Практическое задание по занятию 2 - Практическое применение Docker.

Bookmark

Практическое задание по занятию 2 - Практическое применение Docker.

Задача 0

1. Убедитесь что у вас НЕ(!) установлен `docker-compose`, для этого получите следующую ошибку от команды `docker-compose --version`

```
Command 'docker-compose' not found, but can be installed with:

sudo snap install docker          # version 24.0.5, or
sudo apt  install docker-compose  # version 1.25.0-1

See 'snap info docker' for additional versions.
```

В случае наличия установленного в системе `docker-compose` - удалите его.
2. Убедитесь что у вас УСТАНОВЛЕН `docker compose` (без тире) версии не менее v2.24.X, для это выполните команду `docker compose version`

```
vboxuser@KVA:~$ docker-compose --version
Command 'docker-compose' not found, but can be installed with:
sudo snap install docker          # version 28.4.0, or
sudo apt  install docker-compose  # version 1.29.2-6
See 'snap info docker' for additional versions.
vboxuser@KVA:~$ docker compose version
Docker Compose version v5.1.3
vboxuser@KVA:~$
```

Задача 1

1. Сделайте в своем GitHub пространстве fork репозитория.
2. Создайте файл `Dockerfile.python` на основе существующего `Dockerfile`:
   - Используйте базовый образ `python:3.12-slim`
   - Обязательно используйте конструкцию `COPY . .` в Dockerfile
   - Создайте `.dockerignore` файл для исключения ненужных файлов
   - Используйте `CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5000"]` для запуска

- Протестируйте корректность сборки 2.1 Используйте multistage сборку вместо single stage.

3. (Необязательная часть, *) Изучите инструкцию в проекте и запустите web-приложение без использования docker, с помощью venv. (Mysql БД можно запустить в docker run).
4. (Необязательная часть, *) Изучите код приложения и добавьте управление названием таблицы через ENV переменную.

Детальный разбор задачи 1

1. В правом верхнем углу нажал кнопку **Fork**, выбрал предложенный путь: свой акк, репозиторий с названием как у источника.
2. Сделал git clone https://github.com/ArturP1rozhkov/shvirtd-example-python.git на свой комп
3. Проверил (`ls -la`), перешел в папку `cd shvirtd-example-python`, проверил ветку (`git status` - main)
4. смотрим что за зависимости нужно скачать (`cat requirements.txt`), вывод потока в stdout первых 80 строк скрипта python для проверки наличия объекта app который необходимо запускать, импорты, конфигурация и др. (`sed -n '1,80p' main.py`), вывод содержимого оригинального текста файла dockerfile (`cat Dockerfile`)
5. требования к `Dockerfile.python`:
   - базовый образ должен быть `python:3.12-slim`, это зашито в исходныя Dockerfile.
   - скопировать код приложения в образ.
   - установить зависимости из `requirements.txt`, иначе приложение не запустится.
   - запускать контейнер через `CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5000"]`.
   - multistage - компактность образа на финальном этапе. Два `FROM`: отдельный этап для сборки/установки зависимостей и отдельный для окончательного runtime-образа.

Создаем Dockerfile:

```
# Первый этап:  берем готовый базовый образ Python 3.12 в slim-варианте, даем ему имя buildrer для последующего копирования из него файлов ов второй этап
FROM python:3.12-slim AS builder

```

```
# Задаем рабочую директорию внутри контейнера для последующих инструкций.
WORKDIR /app

# копируем файлы сборки из текущей директории в файловую систему образа
COPY . .

# примечание 1 ниже блока с кодом
RUN pip install --no-cache-dir --upgrade pip && \
     pip install --no-cache-dir --prefix=/install -r requirements.txt

# новый этап сборки с чистого базового образа python:3.12-slim
FROM python:3.12-slim

# снова задаём рабочую директорию, поскольку контекст предыдущего stage не солхранен
WORKDIR /app

# Копируем зависимости из builder в /usr/local - стандартную директорию, где находятся бинарники python
COPY --from=builder /install /usr/local
# копируем сам код проекта
COPY . .
# команда, которая будет выполнена при запуске контейнера. 0.0.0.0 означает «слушать на всех интерфейсах контейнера», чтобы доступ к приложению работал через проброс порта docker run -p
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5000"]


```

Примечание 1 по установке зависимостей в образ:

применение конструкции `pip install --no-cache-dir --upgrade pip` описано в документации к докеру (https://docs.docker.com/guides/sentiment-analysis/ п.4). `--no-cache-dir` опция отключает кэш, что уменьшает размер образа Docker, не сохраняя ненужные данные кэша при обновлении pip. `pip install --no-cache-dir --prefix=/install -r requirements.txt` - устанавливаем зависимости из файла `requirements.txt`, но **не в стандартное системное место**, а в специальный каталог `/install`. То есть зависимости ставятся не прямо в `/usr/local/...`, а в специальный каталог `/install`, который потом будет удобно целиком скопировать во второй stage. В конструкции `RUN pip install -r requirements.txt` пакеты установились бы сразу в системное окружение текущего образа. Но поскольку нам нужны только результаты установки

зависимостей, мы складываем зависимости в отдельное место `/install`, а потом во втором этапе делаем: `COPY --from=builder /install /usr/local`, и таким образом переносим готовую установленную Python-среду из builder в финальный runtime-образ.

Создаем файл Dockerignore

```
.git
.gitignore #будет создан позже
__pycache__ #будет создан позже после запуска python
*.pyc #будет создан позже после запуска python
*.pyo #будет создан позже после запуска python
*.log #будет создан позже после запуска python
.env
Dockerfile
Dockerfile.python
haproxy
nginx
proxy.yaml
README.md
LICENSE
schema.pdf
```

В образ попадут только `main.py` и `requirements.txt`

Запускаем сборку образа

```
# тегируем образ тегом shvirtd-python, указываем явно на название файла dockerfile для сборки образа, роскольку название нестандартное
docker build -t shvirtd-python -f Dockerfile.python .
```

Сборка образа:

```
vboxuser@KVA:~/shvirtd-example-python$ docker images | grep shvirtd-python
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
shvirtd-python:latest                3eca16b3b3e9        375MB         88.6MB
vboxuser@KVA:~/shvirtd-example-python$
```

Тестируем корректность сборки:

```
docker run --rm -p 5000:5000 shvirtd-python #--rm - автоматически удалить контейнер после остановки
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker run --rm -p 5000:5000 shvirtd-python
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
^CINFO:     Shutting down
INFO:     Waiting for application shutdown.
INFO:     Application shutdown complete.
INFO:     Finished server process [1]
Приложение запускается...
Ошибка при создании таблицы: 2003 (HY000): Can't connect to MySQL server on '127.0.0.1:3306' (111)
БД недоступна при старте. Таблица будет создана при первом запросе.
Приложение останавливается.
vboxuser@KVA:~/shvirtd-example-python$
```

Необязательная часть: запускаем web-приложение без использования docker, с помощью venv. Mysql БД запускаем в docker run

```
docker run -d \                                  # запуск в фоне - detached
  --name mysql-local \                           # имя контейнера
  -e MYSQL_ROOT_PASSWORD=root \                  # пароль root-пользователя MySQL
  -e MYSQL_DATABASE=example \                    # создаем базу данных exemple
  -e MYSQL_USER=app \                            # создаем пользователя app с паролем very_strong
  -e MYSQL_PASSWORD=very_strong \
  -p 3306:3306 \                                 # проброс порта из контейнера с MySQL наружу
  mysql:8.0                                      # версия образа MySQL, скачается автоматически
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker run -d \
  --name mysql-local \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=example \
  -e MYSQL_USER=app \ …
4f7cfd23b111: Pull complete
c689d1836928: Pull complete
fe997c72f3e9: Pull complete
139627f60883: Pull complete
38fe0e0986de: Pull complete
09b552d72653: Pull complete
a0db31010df3: Pull complete
b723faee7585: Pull complete
5ea08437b3cf: Download complete
cc9a862b4c10: Download complete
Digest: sha256:d0304ed9fdb64a3f6c7ad11a5fb4f13abfc10e6dfa3f288d652e7320c34df7f9
Status: Downloaded newer image for mysql:8.0
2d2653fa0075d22410a05d54b0070889f2e84bb1964c57eb6d9705292666e9fc
vboxuser@KVA:~/shvirtd-example-python$ docker ps | grep mysql-local
2d2653fa0075   mysql:8.0   "docker-entrypoint.s…"   17 seconds ago   Up 15 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysq
l-local
```

В корне проекта создаем папку venv / с изолированным Python окружением (предварительно скачав его в свою ubuntu), активируем его, устанавливаем зависимости вручную:

```
python3 -m venv venv
source venv/bin/activate
```

```
pip install -r requirements.txt
```

далее, берем значения переменных среды окружения из кода main.py:

```
db_host = os.environ.get('DB_HOST', '127.0.0.1')
db_user = os.environ.get('DB_USER', 'app')
db_password = os.environ.get('DB_PASSWORD', 'very_strong')
db_name = os.environ.get('DB_NAME', 'example')
```

и передаем их приложению

```
export DB_HOST='127.0.0.1'
export DB_USER='app'
export DB_PASSWORD='very_strong'
export DB_NAME='example'
```

проверяем через команду `echo` с соответствующим аргументом
Запускаем командой из инструкции:

```
uvicorn main:app --host 0.0.0.0 --port 5000 --reload
```

```
(venv) vboxuser@KVA:~/shvirtd-example-python$ uvicorn main:app --host 0.0.0.0 --port 5000 --reload
INFO:     Will watch for changes in these directories: ['/home/vboxuser/shvirtd-example-python']
INFO:     Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
INFO:     Started reloader process [12436] using WatchFiles
INFO:     Started server process [12438]
INFO:     Waiting for application startup.
Приложение запускается...
Соединение с БД установлено и таблица 'requests' готова к работе.
INFO:     Application startup complete.
```

из нового окна терминала обращаемся по адресу приложения с портами 5000 и 8090, и смотрим что записано в базу:

```
vboxuser@KVA:~/shvirtd-example-python$ curl http://localhost:5000
"TIME: 2026-04-25 13:26:31, IP: похоже, что вы направляете запрос в неверный порт(например curl http://127.0.0.1:5000). Правильное
выполнение задания - отправить запрос в порт 8090."vboxuser@KVA:~/shvirtd-example-python$
vboxuser@KVA:~/shvirtd-example-python$ curl http://localhost:5000/requests
{"total_records":3,"records":[{"id":3,"request_date":"2026-04-25 13:26:31","request_ip":null},{"id":2,"request_date":"2026-04-25 13
:23:21","request_ip":null},{"id":1,"request_date":"2026-04-25 13:18:16","request_ip":null}]}vboxuser@KVA:~/shvirtd-example-python$
vboxuser@KVA:~/shvirtd-example-python$ curl http://localhost:8090
curl: (7) Failed to connect to localhost port 8090 after 0 ms: Couldn't connect to server
vboxuser@KVA:~/shvirtd-example-python$
```

Необязательная часть: управление названием таблицы через env переменную:

Название таблицы зашито в следующих строках кода в файле main.py:
`db_name = os.environ.get('DB_NAME', 'example')`
`CREATE TABLE IF NOT EXISTS {db_name}.requests`
можно следующей строкой после `db_name = os.environ.get('DB_NAME', 'example')` добавить имя таблицы в виде новой переменной `table_name = os.environ.get('TABLE_NAME', 'requests')` и в функции `ensure_table_exists` заменить: `CREATE TABLE IF NOT EXISTS`

`{db_name}.requests (` на `CREATE TABLE IF NOT EXISTS {db_name}.{table_name} (`
тогда в список переменных окружения, ранее взятых из main.py и заданных командой `export` можно добавить переменную с названием таблицы: `export TABLE_NAME='name_of_table'`

Задача 2

1. Создайте в yandex cloud container registry с именем "test" с помощью "yc tool" . Инструкция
2. Настройте аутентификацию вашего локального docker в yandex container registry.
3. Соберите и залейте в него образ с python приложением из задания №1.
4. Просканируйте образ на уязвимости.
5. В качестве ответа приложите отчет сканирования.

Детальный разбор задачи 2

1. Документация по адресу: https://yandex.cloud/ru/docs/container-registry/tutorials/image-auto-scan/console#cli_1.
   Создайте в yandex cloud container registry с именем "test" с помощью "yc tool"

```
yc container registry create --name test
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc container registry create --name test
done (1s)
id: crpicbm0tnqi4bbjujq8
folder_id: b1grsjmidldjs2rmkgd0
name: test
status: ACTIVE
created_at: "2026-04-25T14:28:43.982Z"
```

```
yc container registry configure-docker
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc container registry configure-docker
docker configured to use yc --profile "default" for authenticating "cr.yandex" container registries
Credential helper is configured in '/home/vboxuser/.docker/config.json'
```

Собираем образ с данными из ответов ya_cli выше:

```
docker build -t cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app .
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker build -t cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app .
[+] Building 1.4s (6/6) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 280B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                           0.9s
 => [auth] library/python:pull token for registry-1.docker.io                                                                 0.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 176B                                                                                             0.0s
 => CACHED [1/1] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3  0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3        0.0s
 => exporting to image                                                                                                        0.2s
 => => exporting layers                                                                                                       0.0s
 => => exporting manifest sha256:9ef88785f0c3755030627034ca6948eadf9f71a5c0b7bbb6126f5adb49521318                             0.0s
 => => exporting config sha256:75cdcac17ea7b30e5d7c0c92f7ac285cd05b6cdcac42bbc8ea62cf734a35cda6                               0.0s
 => => exporting attestation manifest sha256:7e99b0364511f1c773d240cb3ed8bc160a4884a454018e0630e0a94504dcd1fb                 0.0s
 => => exporting manifest list sha256:2d98ec64d7663aa9d22d7a7cf370db1d6ea36b4085bfcb056597405a4d674080                         0.0s
 => => naming to cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app                                                                 0.0s
 => => unpacking to cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app                                                              0.0s
vboxuser@KVA:~/shvirtd-example-python$
```

пушим образ в Yandex Cloud

```
docker push cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker push cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app
The push refers to repository [cr.yandex/crpicbm0tnqi4bbjujq8/my_app]
4d873bcef452: Pushed
3531af2bc2a9: Pushed
ec68dfc1dbd3: Pushed
3fc9d9ab5045: Pushed
cb8c0a9140ac: Pushed
my_app: digest: sha256:2d98ec64d7663aa9d22d7a7cf370db1d6ea36b4085bfcb056597405a4d674080 size: 856
vboxuser@KVA:~/shvirtd-example-python$
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc container image list --registry-id crpicbm0tnqi4bbjujq8
+----------------------+----------------------+----------------------------+----------------------------------------------------------------
---+----------------+
|          ID          |       CREATED        |            NAME            |                                TAGS                                |
   | COMPRESSED SIZE |
+----------------------+----------------------+----------------------------+----------------------------------------------------------------
---+----------------+
| crpcf4p7sv8h5qjr76mg | 2026-04-25 14:42:12 | crpicbm0tnqi4bbjujq8/my_app | my_app
   | 41.2 MB        |
| crpcki0v70ov4aoodb2j | 2026-04-25 14:42:12 | crpicbm0tnqi4bbjujq8/my_app | sha256:7e99b0364511f1c773d240cb3ed8bc160a4884a454018e0630e0a94504dcd1
fb | 1.4 KB         |
| crp1hq1pr4gmacc437ha | 2026-04-25 14:42:11 | crpicbm0tnqi4bbjujq8/my_app | sha256:9ef88785f0c3755030627034ca6948eadf9f71a5c0b7bbb6126f5adb495213
18 | 41.2 MB        |
+----------------------+----------------------+----------------------------+----------------------------------------------------------------
---+----------------+
```

Создаю сервисный аккаунт с именем scanner и назначаю ему роль на свой каталог id b1grsjmidldjs2rmkgd0

```
yc iam service-account create --name scanner

yc resource-manager folder add-access-binding b1grsjmidldjs2rmkgd0 \
--role container-registry.images.scanner \
--subject serviceAccount:ajec3mm6jkn7u1eaum4p
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc resource-manager folder add-access-binding b1grsjmidldjs2rmkgd0 \
--role container-registry.images.scanner \
--subject serviceAccount:ajec3mm6jkn7u1eaum4p
done (2s)
effective_deltas:
  - action: ADD
    access_binding:
      role_id: container-registry.images.scanner
      subject:
        id: ajec3mm6jkn7u1eaum4p
        type: serviceAccount
```

запускаю сканирование

```
yc container image scan crpcf4p7sv8h5qjr76mg --impersonate-service-account-id ajec3mm6jkn7u1eaum4p
```

Сканирование не пошло потому, что созданный образ контейнера не поддерживается yandex cloud.

```
ERROR: rpc error: code = FailedPrecondition desc = Image media type is not supported: application/vnd.oci.image.index.v1+json
```

Да, ошибка пофиксена и решение найдено при помощи Claude Sonnet 4.6. В документации Yandex Cloude упоминание такого не нашел, есть в документации на https://docs.docker.com, где их интерактивный бот-помощник при вопросе как пофиксить ошибку, советует при сборке отключить `sbom` *и аттестации происхождения...* Пересобрал контейнер в формате **Docker Manifest v2** с флагами:

```
--provenance=false # отключает attestation-манифест (именно он создаёт OCI Index)
--sbom=false # отключает Software Bill of Materials манифест
# В результате образ будет в формате application/vnd.docker.distribution.manifest.v2+json — его Yandex умеет сканировать.
```

```
# удалил старый образ
yc container image delete crpcf4p7sv8h5qjr76mg

# пересобрал с отключенным provenance и sbom
DOCKER_BUILDKIT=1 docker build \
  --provenance=false \
  --sbom=false \
  -t cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app .

  # запушил снова
docker push cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app

# новый id
yc container image list --registry-id crpicbm0tnqi4bbjujq8
```

```
vboxuser@KVA:~/shvirtd-example-python$ DOCKER_BUILDKIT=1 docker build \
  --provenance=false \
  --sbom=false \
  -t cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app .
vboxuser@KVA:~/shvirtd-example-python$ docker push cr.yandex/crpicbm0tnqi4bbjujq8/my_app:my_app
The push refers to repository [cr.yandex/crpicbm0tnqi4bbjujq8/my_app]
4d873bcef452: Layer already exists
3fc9d9ab5045: Layer already exists
3531af2bc2a9: Layer already exists
cb8c0a9140ac: Layer already exists
my_app: digest: sha256:a535c0acb497f59b173cb9731b77becaa63d0cdee87fca3f6b41882663a1bf7b size: 1095
vboxuser@KVA:~/shvirtd-example-python$ yc container image list --registry-id crpicbm0tnqi4bbjujq8
+----------------------+----------------------+-----------------------------+-----------------------------------------------------------------------+-----------------+
|          ID          |       CREATED        |            NAME             |                                  TAGS                                 | COMPRESSED SIZE |
+----------------------+----------------------+-----------------------------+-----------------------------------------------------------------------+-----------------+
| crp0gt4s970vohnme31t | 2026-04-25 15:13:56 | crpicbm0tnqi4bbjujq8/my_app | my_app                                                                | 41.2 MB         |
| crpcki0v70ov4aoodb2j | 2026-04-25 14:42:12 | crpicbm0tnqi4bbjujq8/my_app | sha256:7e99b0364511f1c773d240cb3ed8bc160a4884a454018e0630e0a94504dcd1fb | 1.4 KB          |
| crp1hq1pr4gmacc437ha | 2026-04-25 14:42:11 | crpicbm0tnqi4bbjujq8/my_app | sha256:9ef88785f0c375503062703 4ca6948eadf9f71a5c0b7bbb6126f5adb49521318 | 41.2 MB         |
+----------------------+----------------------+-----------------------------+-----------------------------------------------------------------------+-----------------+
```

```
yc container image scan crp0gt4s970vohnme31t --impersonate-service-account-id ajec3mm6jkn7u1eaum4p
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc container image scan crp0gt4s970vohnme31t --impersonate-service-account-id ajec3mm6jkn7u1eaum4p
done (1m1s)
id: cheopc0ciouikk81n7h3
image_id: crp0gt4s970vohnme31t
scanned_at: "2026-04-25T15:18:24.384Z"
status: READY
vulnerabilities:
  high: "6"
  medium: "29"
  low: "72"
  undefined: "1"
```

сохранил в файл

```
yc container image list-vulnerabilities \
  --scan-result-id cheopc0ciouikk81n7h3 \
  --impersonate-service-account-id ajec3mm6jkn7u1eaum4p \
  > scan_report.txt
```

```
vboxuser@KVA:~/shvirtd-example-python$ yc container image list-vulnerabilities \
  --scan-result-id cheopc0ciouikk81n7h3 \
  --impersonate-service-account-id ajec3mm6jkn7u1eaum4p \
  > scan_report.txt
```

Задача 3

1. Изучите файл "proxy.yaml"
2. Создайте в репозитории с проектом файл `compose.yaml`. С помощью директивы "include" подключите к нему файл "proxy.yaml".
3. Опишите в файле `compose.yaml` следующие сервисы:

- `web`. Образ приложения должен ИЛИ собираться при запуске compose из файла `Dockerfile.python` ИЛИ скачиваться из yandex cloud container registry(из задание №2 со *). Контейнер должен работать в bridge-сети с названием `backend` и иметь фиксированный ipv4-адрес `172.20.0.5`. Сервис должен всегда перезапускаться в случае ошибок. Передайте необходимые ENV-переменные для подключения к Mysql базе данных по сетевому имени сервиса `web`
- `db`. image=mysql:8. Контейнер должен работать в bridge-сети с названием `backend` и иметь фиксированный ipv4-адрес `172.20.0.10`. Явно перезапуск сервиса в случае ошибок. Передайте необходимые ENV-переменные для создания: пароля root пользователя, создания базы данных, пользователя и пароля для web-приложения.Обязательно используйте уже существующий .env file для назначения секретных ENV-переменных!

2. Запустите проект локально с помощью docker compose , добейтесь его стабильной работы: команда `curl -L http://127.0.0.1:8090` должна возвращать в качестве ответа время и локальный IP-адрес. Если сервисы не

стартуют воспользуйтесь командами: `docker ps -a` и `docker logs <container_name>` . Если вместо IP-адреса вы получаете информационную ошибку --убедитесь, что вы шлете запрос на порт `8090`, а не 5000.

3. Подключитесь к БД mysql с помощью команды `docker exec -ti <имя_контейнера> mysql -uroot -p<пароль root-пользователя>`(обратите внимание что между ключем -u и логином root нет пробела. это важно!!! тоже самое с паролем) . Введите последовательно команды (не забываем в конце символ ;): `show databases; use <имя вашей базы данных(по-умолчанию virtd, как это указано в .env)>; show tables; SELECT * from requests LIMIT 10;` . Примечание: таблица в БД создается после первого поступившего запроса к приложению.
4. Остановите проект. В качестве ответа приложите скриншот sql-запроса.

Детальный разбор задачи 3:

файл proxy.yaml `vboxuser@KVA:~/shvirtd-example-python$ cat proxy.yaml`

```yaml
version: '3.8'
services:

  reverse-proxy:
    image: haproxy:2.4
    restart: always
    networks:
        backend: {}
    ports:
    - "127.0.0.1:8080:8080"
    volumes:
    - ./haproxy/reverse/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:rw

  ingress-proxy:
    image: nginx:latest
    restart: always
    network_mode: host
    volumes:
    - ./nginx/ingress/default.conf:/etc/nginx/conf.d/default.conf:rw
    - ./nginx/ingress/nginx.conf:/etc/nginx/nginx.conf:rw

networks:
  backend:
    driver: bridge
    ipam:
     config:
```

```
        - subnet: 172.20.0.0/24
```

- **HAProxy** (High Availability Proxy) - это балансировщик нагрузки и прокси-сервер. Он принимает входящие запросы и распределяет их между бэкенд-серверами. В нашем случае он работает **внутри сети backend** и проксирует трафик к сервису `web` на порту `8080`. Конфиг Haproxy находится во внешнем volumes - ./haproxy/reverse/haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:rw.
- **Ingress-proxy** — это точка входа трафика извне в систему. Nginx здесь выступает как **реверс-прокси** - принимает HTTP-запросы из внешнего мира и передаёт их дальше в HAProxy. Nginx лучше справляется с задачами приёма внешнего трафика. HAProxy лучше для внутренней балансировки между несколькими бэкенд сервисами. Инструкция `network_mode: host` обозначает что Nginx **не имеет своей сети** — он напрямую использует сеть хоста. Поэтому он может слушать порт `8090` хоста без проброса портов. Настройки - через конфиги во внешних volumes: `./nginx/ingress/default.conf:/etc/nginx/conf.d/default.conf:rw` и `./nginx/ingress/nginx.conf:/etc/nginx/nginx.conf:rw`
- **Сеть backend** (виртуальная изолированная сеть на одном хосте) назначена явно: 172.20.0.0/24.

Создаем в локальном репозитории (в папке проекта) файл compose.yaml

```
nano compose.yaml
```

вносим туда:

```
include:
  - proxy.yaml

services:
  web:
    build:
      context: .
      dockerfile: Dockerfile.python
    restart: always
    env_file:
      - .env
    environment:
      - DB_HOST=db
      - DB_NAME=${MYSQL_DATABASE}
      - DB_USER=${MYSQL_USER}
```

```
16        - DB_PASSWORD=${MYSQL_PASSWORD}
17      networks:
18        backend:
19          ipv4_address: 172.20.0.5
20
21    db:
22      image: mysql:8
23      restart: always
24      env_file:
25        - .env
26      environment:
27        - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
28        - MYSQL_DATABASE=${MYSQL_DATABASE}
29        - MYSQL_USER=${MYSQL_USER}
30        - MYSQL_PASSWORD=${MYSQL_PASSWORD}
31      networks:
32        backend:
33          ipv4_address: 172.20.0.10
```

- `include`: - proxy.yaml- подключили файл proxy.yaml со всеми его содержимым и настройками сервисов, а также сетью backend.
- секция `web` в файле означает что она будет резолвиться внутри сети как "web"
- секция `build` назначает источник сборки - Dockerfile.python - нестандартный файл в контексте рабочей директории `.` (иначе было бы просто `build: .`)
- `env_file`: - `.env` - откуда брать переменные среды.
- `environment` - какие именно переменные передаем конкретному контейнеру: `web` - сервису, поднимаемому `docktrfile.python` или `db` из образа `mysql:8`. В первом случае, переменные описаны понятным языком для python, во втором - для mysql
- секция networks: сервис `web` (контейнер с приложением python из `dockerfile.python`) будет жить по адресу 172.20.0.5 из диапазона сети `backend`, явно прописаного в файле `proxy.yaml`. Аналогично - `db` всегда будет `172.20.0.10`

Проверяем конфиг compose, чтобы не было ошибок, затем запускаем все это. Проверяем чтобы все 4 контейнера работали:

```
1  docker compose config # проверка конфига
2  docker compose up -d  # поднимаем сервис в фоновом режиме (detached
   mode)
```

```
docker ps                # проверяем что все контейнеры работают
```

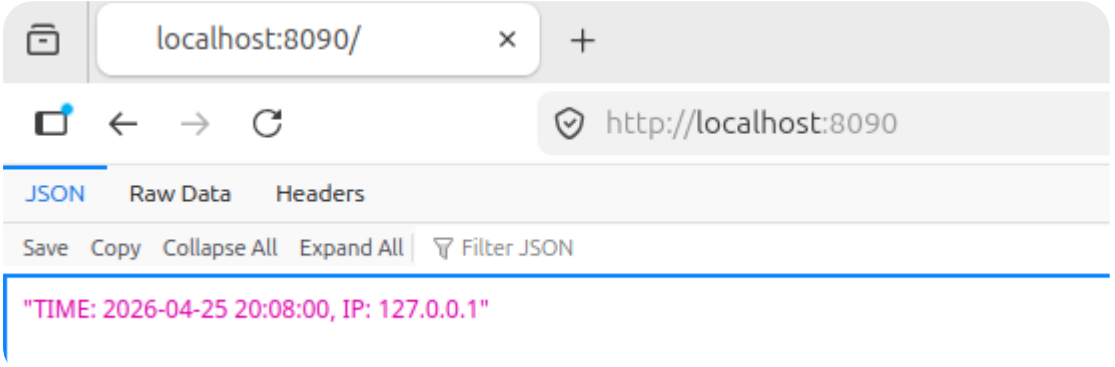
```
vboxuser@KVA:~/shvirtd-example-python$ docker compose up -d
 ✔ Image haproxy:2.4  Pulled                                                                                                  7.0s
 ✔ Image mysql:8      Pulled                                                                                                 21.0s
[+] Building 35.8s (14/14) FINISHED
 => [internal] load local bake definitions                                                                                    0.0s
 => => reading from stdin 551B                                                                                                0.0s
 => [internal] load build definition from Dockerfile.python                                                                   0.0s
 => => transferring dockerfile: 379B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                           1.0s
 => [auth] library/python:pull token for registry-1.docker.io                                                                 0.0s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 192B                                                                                             0.0s
 => [builder 1/4] FROM docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3  0.1s
 => => resolve docker.io/library/python:3.12-slim@sha256:46cb7cc2877e60fbd5e21a9ae6115c30ace7a077b9f8772da879e4590c18c2e3         0.1s
 => [internal] load build context                                                                                             2.8s
 => => transferring context: 161.81MB                                                                                         2.8s
 => CACHED [builder 2/4] WORKDIR /app                                                                                         0.0s
 => [builder 3/4] COPY . .                                                                                                    2.0s
 => [builder 4/4] RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-dir --prefix=/install -r requirements.txt  12.5s
 => [stage-1 3/4] COPY --from=builder /install /usr/local                                                                     0.5s
 => [stage-1 4/4] COPY . .                                                                                                    1.6s
 => exporting to image                                                                                                       13.8s
 => => exporting layers                                                                                                       9.0s
 => => exporting manifest sha256:7eaceba48954d760e6d0565e32180bd55b500a35cf5f9ecad2101608bf081ba1                             0.0s
 => => exporting config sha256:da5be2d9d6b9ae2246f54a5b45709aea556e741a2e4cf58ebfbaeea66c4c278d                               0.0s
 => => exporting attestation manifest sha256:6216a84e0f1d2d1ce9e494e6221a7fb0ad080cd1515982eaf0642b53e5b9f421                 0.0s
[+] up 34/34ting manifest list sha256:e9c5ac1e33de8b28a966d6fb7e9f893dd49715464eb10a5ce1a4940549354959                        0.0s
 ✔ Image nginx:latest                                Pulled                                                                  12.4s
 ✔ Image haproxy:2.4                                 Pulled                                                                   7.0s
 ✔ Image mysql:8                                     Pulled                                                                  21.0s
 ✔ Image shvirtd-example-python-web                  Built                                                                   35.9s
 ✔ Network shvirtd-example-python_backend            Created                                                                  0.1s
 ✔ Container shvirtd-example-python-ingress-proxy-1  Started                                                                  1.5s
 ✔ Container shvirtd-example-python-reverse-proxy-1  Started                                                                  1.8s
 ✔ Container shvirtd-example-python-web-1            Started                                                                  1.8s
 ✔ Container shvirtd-example-python-db-1             Started                                                                  1.8s
vboxuser@KVA:~/shvirtd-example-python$ docker ps
CONTAINER ID   IMAGE                      COMMAND                  CREATED          STATUS          PORTS                                          NAMES
e7559a3f2b49   nginx:latest               "/docker-entrypoint.…"   19 seconds ago   Up 17 seconds                                                  shvirtd-example-python-in
gress-proxy-1
a35ed317b097   haproxy:2.4                "docker-entrypoint.s…"   19 seconds ago   Up 17 seconds   127.0.0.1:8080->8080/tcp                       shvirtd-example-python-re
verse-proxy-1
352b12d556f0   mysql:8                    "docker-entrypoint.s…"   19 seconds ago   Up 17 seconds   3306/tcp, 33060/tcp                            shvirtd-example-python-db
-1
af172876c8e5   shvirtd-example-python-web "uvicorn main:app --…"   19 seconds ago   Up 17 seconds                                                  shvirtd-example-python-we
b-1
2d2653fa0075   mysql:8.0                  "docker-entrypoint.s…"   7 hours ago      Up 7 hours      0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp   mysql-local
vboxuser@KVA:~/shvirtd-example-python$
```

Проверяем доступность:

```
curl -L http://127.0.0.1:8090
```

```
vboxuser@KVA:~/shvirtd-example-python$ curl -L http://127.0.0.1:8090
"TIME: 2026-04-25 20:07:33, IP: 127.0.0.1"vboxuser@KVA:~/shvirtd-example-python$
```

в браузере:



Подключаемся к базе данных mySQL (контейнер shvirtd-example-python-db-1 из вывода команды docker ps)

```
docker exec -ti shvirtd-example-python-db-1 mysql -uroot -pYtReWq4321
```

вводим последовательно команды (по одной)

```
show databases;
use virtd;
```

```sql
show tables;
SELECT * FROM requests LIMIT 10;
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker exec -ti shvirtd-example-python-db-1 mysql -uroot -pYtReWq4321
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 12
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| virtd              |
+--------------------+
5 rows in set (0.01 sec)

mysql> use virtd;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----------------+
| Tables_in_virtd |
+-----------------+
| requests        |
+-----------------+
1 row in set (0.01 sec)

mysql> SELECT * FROM requests LIMIT 10;
+----+---------------------+------------+
| id | request_date        | request_ip |
+----+---------------------+------------+
|  1 | 2026-04-25 20:07:33 | 127.0.0.1  |
|  2 | 2026-04-25 20:08:00 | 127.0.0.1  |
+----+---------------------+------------+
2 rows in set (0.00 sec)

mysql>
```

Изменения запушил в мой форк учебного репозитория на github

Задача 4

1. Запустите в Yandex Cloud ВМ (вам хватит 2 Гб Ram).
2. Подключитесь к Вм по ssh и установите docker.
3. Напишите bash-скрипт, который скачает ваш fork-репозиторий в каталог /opt и запустит проект целиком.
4. Зайдите на сайт проверки http подключений, например(или аналогичный): `https://check-host.net/check-http` и запустите проверку вашего сервиса `http://<внешний_IP-адрес_вашей_ВМ>:8090`. Таким образом трафик будет направлен в ingress-proxy. Трафик должен пройти через цепочки: Пользователь → Internet → Nginx → HAProxy → FastAPI(запись в БД) → HAProxy → Nginx → Internet → Пользователь
5. (Необязательная часть) Дополнительно настройте remote ssh context к вашему серверу. Отобразите список контекстов и результат удаленного выполнения `docker ps -a`
6. Повторите SQL-запрос на сервере и приложите скриншот и ссылку на fork.

Детальный разбор задачи 4:

1. Поднял ВМ `Ubuntu 22.04 LTS OS Login` в Yandex Cloud с параметрами:

Ресурсы

Платформа	Intel Ice Lake
Гарантированная доля vCPU	20%
vCPU	2
RAM	2 ГБ
Объём дискового пространства	10 ГБ
Прерываемая	Да

2. Подключился со своей хостовой ВМ по SSH и установил Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
```

```
kva@compute-vm-2-2-10-hdd-1777199987547:~$ docker --version
Docker version 29.4.1, build 055a478
kva@compute-vm-2-2-10-hdd-1777199987547:~$ docker compose version
Docker Compose version v5.1.3
kva@compute-vm-2-2-10-hdd-1777199987547:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
kva@compute-vm-2-2-10-hdd-1777199987547:~$
```

1. Поскольку, мой проект со всеми изменениями запушен в фок учебного репозиторя на моем Github, то bash скрипт клонирования проекта в каталог /opt на ВМ Яндекс Клауд будет выглядеть так:

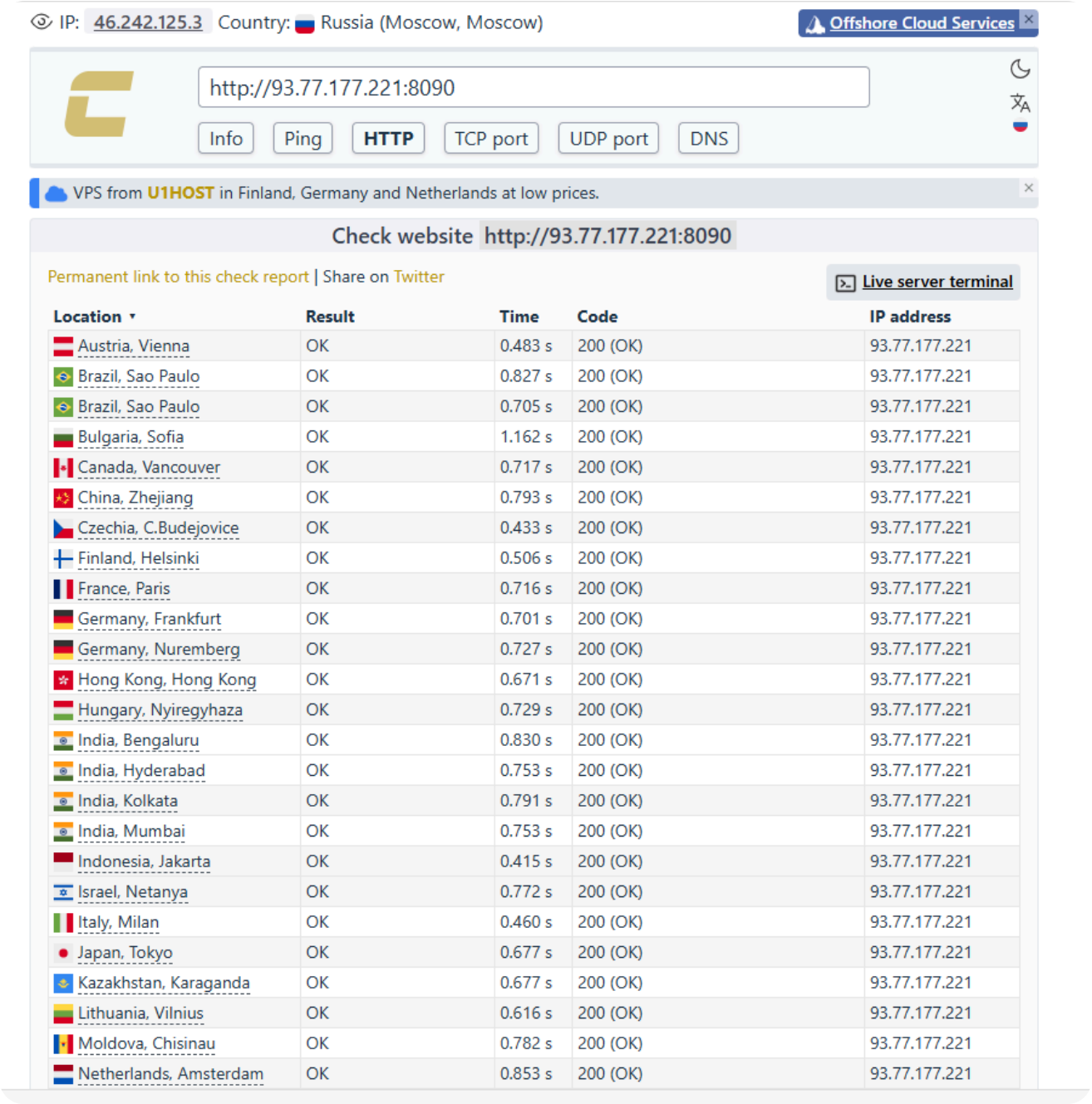
```
#!/bin/bash
set -e # инструкция о том, что необходимо остановить скрипт при ошибке на любом этапе
git clone https://github.com/ArturP1rozhkov/shvirtd-example-python /opt/shvirtd-example-python  # клонируем репозиторий в нужную папку
cd /opt/shvirtd-example-python # переходим в папку проекта
docker compose up -d           # запускаем сценарий проекта
docker ps                      # проверяем как все запустилось
```

Единственной проблемой является то, что при создании форка в мой Github репозиторий из родительского попал файл .env с секретами, что недопустимо с точки зрения инфобеза. В реальном сценарии его нахождение в репозитории проекта недопустимо, а передача секретов должна происходить иным способом.

2. Добавляю скрипт в файл: `sudo nano /opt/script.sh`, делаю его исполняемым `sudo chmod +x /opt/script.sh`, запускаю: `sudo /opt/script.sh`

```
CONTAINER ID   IMAGE                      COMMAND                  CREATED        STATUS         PORTS                      NAMES
ef7bb87aeab3   nginx:latest               "/docker-entrypoint.…"   6 seconds ago  Up 2 seconds                              shvirtd-example-python-ingress-proxy-1
cae601328faf   haproxy:2.4                "docker-entrypoint.s…"   6 seconds ago  Up 2 seconds   127.0.0.1:8080->8080/tcp   shvirtd-example-python-reverse-proxy-1
2f50392b7073   mysql:8                    "docker-entrypoint.s…"   6 seconds ago  Up 2 seconds   3306/tcp, 33060/tcp        shvirtd-example-python-db-1
122880bfd5a4   shvirtd-example-python-web "uvicorn main:app --…"   6 seconds ago  Up 2 seconds                              shvirtd-example-python-web-1
kva@compute-vm-2-2-10-hdd-1777199987547:~$
```

3. Захожу на https://check-host.net/check-http, где в поле ввода команде вставляю: http://93.77.177.221:8090



Location	Result	Time	Code	IP address
Austria, Vienna	OK	0.483 s	200 (OK)	93.77.177.221
Brazil, Sao Paulo	OK	0.827 s	200 (OK)	93.77.177.221
Brazil, Sao Paulo	OK	0.705 s	200 (OK)	93.77.177.221
Bulgaria, Sofia	OK	1.162 s	200 (OK)	93.77.177.221
Canada, Vancouver	OK	0.717 s	200 (OK)	93.77.177.221
China, Zhejiang	OK	0.793 s	200 (OK)	93.77.177.221
Czechia, C.Budejovice	OK	0.433 s	200 (OK)	93.77.177.221
Finland, Helsinki	OK	0.506 s	200 (OK)	93.77.177.221
France, Paris	OK	0.716 s	200 (OK)	93.77.177.221
Germany, Frankfurt	OK	0.701 s	200 (OK)	93.77.177.221
Germany, Nuremberg	OK	0.727 s	200 (OK)	93.77.177.221
Hong Kong, Hong Kong	OK	0.671 s	200 (OK)	93.77.177.221
Hungary, Nyiregyhaza	OK	0.729 s	200 (OK)	93.77.177.221
India, Bengaluru	OK	0.830 s	200 (OK)	93.77.177.221
India, Hyderabad	OK	0.753 s	200 (OK)	93.77.177.221
India, Kolkata	OK	0.791 s	200 (OK)	93.77.177.221
India, Mumbai	OK	0.753 s	200 (OK)	93.77.177.221
Indonesia, Jakarta	OK	0.415 s	200 (OK)	93.77.177.221
Israel, Netanya	OK	0.772 s	200 (OK)	93.77.177.221
Italy, Milan	OK	0.460 s	200 (OK)	93.77.177.221
Japan, Tokyo	OK	0.677 s	200 (OK)	93.77.177.221
Kazakhstan, Karaganda	OK	0.677 s	200 (OK)	93.77.177.221
Lithuania, Vilnius	OK	0.616 s	200 (OK)	93.77.177.221
Moldova, Chisinau	OK	0.782 s	200 (OK)	93.77.177.221
Netherlands, Amsterdam	OK	0.853 s	200 (OK)	93.77.177.221

4. Подключаюсь к базе данных mySQL на удаленной ВМ

```
docker exec -ti shvirtd-example-python-db-1 mysql -uroot -pYtReWq4321
```

5. Ввожу последовательно команды (по одной)

```
show databases;
use virtd;
show tables;
```

```sql
SELECT * FROM requests LIMIT 10;
```

```
kva@compute-vm-2-2-10-hdd-1777199987547:~$ docker exec -ti shvirtd-example-python-db-1 mysql -uroot -pYtReWq4321
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 78
Server version: 8.4.9 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| virtd              |
+--------------------+
5 rows in set (0.01 sec)

mysql> use virtd;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----------------+
| Tables_in_virtd |
+-----------------+
| requests        |
+-----------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM requests LIMIT 40;
+----+---------------------+-----------------+
| id | request_date        | request_ip      |
+----+---------------------+-----------------+
|  1 | 2026-04-26 12:17:16 | 77.75.230.51    |
|  2 | 2026-04-26 12:17:16 | 185.25.204.60   |
|  3 | 2026-04-26 12:17:16 | 185.221.199.82  |
|  4 | 2026-04-26 12:17:16 | 178.216.200.169 |
|  5 | 2026-04-26 12:17:16 | 185.224.3.111   |
|  6 | 2026-04-26 12:17:16 | 83.143.119.103  |
|  7 | 2026-04-26 12:17:16 | 195.137.245.1   |
|  8 | 2026-04-26 12:17:16 | 78.40.111.151   |
|  9 | 2026-04-26 12:17:16 | 65.109.182.130  |
| 10 | 2026-04-26 12:17:16 | 88.119.179.10   |
| 11 | 2026-04-26 12:17:16 | 81.6.41.187     |
| 12 | 2026-04-26 12:17:16 | 185.83.213.25   |
| 13 | 2026-04-26 12:17:16 | 194.26.100.165  |
| 14 | 2026-04-26 12:17:16 | 195.137.244.1   |
| 15 | 2026-04-26 12:17:16 | 195.154.114.92  |
| 16 | 2026-04-26 12:17:16 | 185.37.147.117  |
| 17 | 2026-04-26 12:17:16 | 194.26.229.20   |
| 18 | 2026-04-26 12:17:16 | 185.130.104.238 |
| 19 | 2026-04-26 12:17:16 | 45.159.248.77   |
| 20 | 2026-04-26 12:17:16 | 116.203.22.31   |
| 21 | 2026-04-26 12:17:16 | 45.9.168.235    |
| 22 | 2026-04-26 12:17:16 | 107.149.201.15  |
| 23 | 2026-04-26 12:17:16 | 78.40.116.61    |
| 24 | 2026-04-26 12:17:16 | 77.92.151.181   |
| 25 | 2026-04-26 12:17:16 | 195.211.27.85   |
| 26 | 2026-04-26 12:17:16 | 185.209.161.145 |
| 27 | 2026-04-26 12:17:16 | 91.231.182.39   |
| 28 | 2026-04-26 12:17:16 | 194.146.57.64   |
| 29 | 2026-04-26 12:17:16 | 185.23.17.21    |
| 30 | 2026-04-26 12:17:16 | 185.86.77.126   |
| 31 | 2026-04-26 12:17:16 | 185.120.77.165  |
| 32 | 2026-04-26 12:17:16 | 147.45.113.30   |
| 33 | 2026-04-26 12:17:16 | 178.17.171.235  |
| 34 | 2026-04-26 12:17:16 | 195.211.24.48   |
| 35 | 2026-04-26 12:17:16 | 147.45.60.14    |
| 36 | 2026-04-26 12:17:16 | 64.72.205.76    |
| 37 | 2026-04-26 12:17:16 | 176.125.231.26  |
| 38 | 2026-04-26 12:17:16 | 157.254.174.204 |
| 39 | 2026-04-26 12:17:16 | 198.135.169.20  |
| 40 | 2026-04-26 12:17:17 | 45.252.248.142  |
+----+---------------------+-----------------+
40 rows in set (0.00 sec)
```

6. Ссылка на форк в мой репозиторий: [https://github.com/ArturP1rozhkov/shvirtd-example-**python**](https://github.com/ArturP1rozhkov/shvirtd-example-python)
7. Настраиваю remote ssh context к удаленному серверу с использованием уже имеющегося ключа SSH (полная инструкция: https://docs.docker.com/reference/cli/docker/context/create/):

```
docker context create yc-kva \                    # создаю новый контекст со своим описанием yc-kva
  --description "Yandex Cloud VM KVA" \           # описание контекста. Необязательный параметр
  --docker "host=ssh://kva@93.77.177.221"         # адрес докер-демона: протокол SSH, логин-kva, публичный ip
```

8. Список контекстов и результат удаленного выполнения `docker ps -a`

```
vboxuser@KVA:~/shvirtd-example-python$ docker context create yc-kva \
    --description "Yandex Cloud VM KVA" \
    --docker "host=ssh://kva@93.77.177.221"
yc-kva
Successfully created context "yc-kva"
vboxuser@KVA:~/shvirtd-example-python$ docker context ls
NAME        DESCRIPTION                               DOCKER ENDPOINT               ERROR
default *   Current DOCKER_HOST based configuration   unix:///var/run/docker.sock
yc-kva      Yandex Cloud VM KVA                       ssh://kva@93.77.177.221
vboxuser@KVA:~/shvirtd-example-python$ docker ps -a
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS                     PORTS                                      NAMES
e7559a3f2b49   nginx:latest                   "/docker-entrypoint.…"   17 hours ago   Up 2 hours                                                            shvirtd-example-python-ingress-
proxy-1
a35ed317b097   haproxy:2.4                    "docker-entrypoint.s…"   17 hours ago   Up 2 hours                 127.0.0.1:8080->8080/tcp                   shvirtd-example-python-reverse-
proxy-1
352b12d556f0   mysql:8                        "docker-entrypoint.s…"   17 hours ago   Up 2 hours                 3306/tcp, 33060/tcp                        shvirtd-example-python-db-1
af172876c8e5   shvirtd-example-python-web     "uvicorn main:app --…"   17 hours ago   Up 2 hours                                                            shvirtd-example-python-web-1
2d2653fa0075   mysql:8.0                      "docker-entrypoint.s…"   24 hours ago   Exited (0) 16 hours ago                                               mysql-local
b9799436e106   127.0.0.1:5000/custom-nginx    "/docker-entrypoint.…"   8 days ago     Exited (255) 3 days ago    0.0.0.0:9090->80/tcp, [::]:9090->80/tcp    custom-nginx-stack-nginx-1
cc3e6edb4c32   debian:latest                  "sleep infinity"         8 days ago     Exited (137) 8 days ago                                               debian-container
9567fa46ac40   almalinux:latest               "sleep infinity"         8 days ago     Exited (137) 8 days ago                                               centos-container
vboxuser@KVA:~/shvirtd-example-python$
```

Задача 5

1. Напишите и задеплойте на вашу облачную ВМ bash скрипт, который произведет резервное копирование БД mysql в директорию "/opt/backup" с помощью запуска в сети "backend" контейнера из образа `schnitzler/mysqldump` при помощи `docker run ...` команды. Подсказка: "документация образа."
2. Протестируйте ручной запуск
3. Настройте выполнение скрипта раз в 1 минуту через cron, crontab или systemctl timer. Придумайте способ не светить логин/пароль в git!!
4. Предоставьте скрипт, cron-task и скриншот с несколькими резервными копиями в "/opt/backup"

Детальный разбор задачи 5:

1. Документация по `schnitzler/mysqldump` находится по адресу: https://hub.docker.com/r/schnitzler/mysqldump/.
2. Контейнер предназначен для регулярного резервного копирования скриптом через crontab.
3. Создаем необходимые директории на удаленной машине и заносим в них данные:

```
sudo mkdir -p /opt/backup                                   # Создаём директорию для резервных копий

sudo nano /opt/backup/.env                                  # Создаём .env файл с переменными окружения и секретами

sudo chmod 600 /opt/backup/.env                             # Выставляем необходимые права

sudo nano /opt/backup/mysql_backup.sh                       # Создаём файл со скриптом

```

```
sudo chmod +x /opt/backup/mysql_backup.sh            # Делаем его исполняемым

sudo /opt/backup/mysql_backup.sh                     # Ручной запуск

ls -la /opt/backup/                                  # Проверка резльтата
```

1. Скрипт `mysql_backup.sh`

```
#!/bin/bash

source /opt/backup/.env                              # источник переменных, путь внутри ВМ, не в git

TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")               # формат записи файла

docker run --rm \                                    # инструкции докера - удалить контейнер после выполнения
  --entrypoint "" \                                  # отключение внутреннего планировщика контейнера
  --network shvirtd-example-python_backend \         # контейнер в той же сети что и MSsql
  -v /opt/backup:/backup \                           # примонтированная папка хоста ВМ
  schnitzler/mysqldump \                             # образ контейнера на докерхаб
  mysqldump --opt \                                  # инструкции утилиты mysqldump - оптимизация
    --no-tablespaces \                               # добавлено для исправления ошибок создания дампа
    -h "${MYSQL_HOST}" \                             # адрес БД из переменных
    -u "${MYSQL_USER}" \                             # имя пользователя из переменных
    -p"${MYSQL_PASSWORD}" \                          # пароль из переменных
    --result-file="/backup/dump_${TIMESTAMP}.sql" \  # куда писать файл дампа в контейнере
    "${MYSQL_DATABASE}"                              # имя БД которую дампим
```

С данным сценарием появляется ошибка авторизации ДБ.

```
mysqldump: Got error: 1045: "Plugin caching_sha2_password could not be loaded: Error loading shared library /usr/lib/mariadb/plugin/caching_sha2_password.so: No such file or directory" when trying to connect
```

Пояснение: Дефолтный способ хранения паролей в MySQL 8 - плагин `caching_sha2_password`.
`schnitzler/mysqldump` собран на основе **MariaDB-клиента**. MariaDB - это форк MySQL, отдельный продукт. Он умеет работать только со старым плагином — `mysql_native_password`.
В файл compose.yaml на ВМ добавил строку с инструкцией принудительно включать плагин:

```
command: --mysql-native-password=ON
```

далее пересоздал контейнеры

```
docker compose down
docker compose up -d
```

вошел в ДБ

```
docker exec -it shvirtd-example-python-db-1 mysql -u root -pYtReWq4321
```

выполнил:

```
ALTER USER 'app'@'%' IDENTIFIED WITH mysql_native_password BY 'QwErTy1234';
FLUSH PRIVILEGES;
```

возникла ошибка

```
mysqldump: Error: 'Access denied; you need (at least one of) the PROCESS privilege(s) for this operation' when trying to dump tablespaces
```

отключил добавлением строки `--no-tablespaces \` в скрипт. Запустил. Дамп создан в ручном режиме.

```
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$ sudo /opt/backup/mysql_backup.sh
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$ ls -la
total 32
drwxr-xr-x 2 root root 4096 Apr 26 16:15 .
drwxr-xr-x 5 root root 4096 Apr 26 15:09 ..
-rw------- 1 root root  115 Apr 26 15:10 .env
-rw-r--r-- 1 root root    0 Apr 26 15:36 dump_2026-04-26_15-36-13.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:03 dump_2026-04-26_16-03-57.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:08 dump_2026-04-26_16-08-10.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:15 dump_2026-04-26_16-15-23.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:15 dump_2026-04-26_16-15-51.sql
-rwxr-xr-x 1 root root  410 Apr 26 16:07 mysql_backup.sh
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$ head -20 /opt/backup/dump_2026-04-26_16-15-51.sql
-- MariaDB dump 10.19  Distrib 10.11.4-MariaDB, for Linux (x86_64)
--
-- Host: db    Database: virtd
-- ------------------------------------------------------
-- Server version       8.4.9

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
/*!40103 SET TIME_ZONE=@OLD_TIME_ZONE */;

/*!40101 SET SQL_MODE=@OLD_SQL_MODE */;
/*!40014 SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS */;
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$
```

вводим скрипт в cron:

```
* * * * * /opt/backup/mysql_backup.sh
```

```
GNU nano 6.2                                                    /tmp/crontab.qPZnEd/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /opt/backup/mysql_backup.sh
```

фиксирую создание дампов в целевой папке:

```
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$ ls -la
total 48
drwxr-xr-x 2 root root 4096 Apr 26 16:24 .
drwxr-xr-x 5 root root 4096 Apr 26 15:09 ..
-rw------- 1 root root  115 Apr 26 15:10 .env
-rw-r--r-- 1 root root    0 Apr 26 15:36 dump_2026-04-26_15-36-13.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:03 dump_2026-04-26_16-03-57.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:08 dump_2026-04-26_16-08-10.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:15 dump_2026-04-26_16-15-23.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:15 dump_2026-04-26_16-15-51.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:21 dump_2026-04-26_16-21-01.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:22 dump_2026-04-26_16-22-01.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:23 dump_2026-04-26_16-23-01.sql
-rw-r--r-- 1 root root 1252 Apr 26 16:24 dump_2026-04-26_16-24-01.sql
-rwxr-xr-x 1 root root  410 Apr 26 16:07 mysql_backup.sh
kva@compute-vm-2-2-10-hdd-1777199987547:/opt/backup$
```

Задача 6

Скачайте docker образ `hashicorp/terraform:latest` и скопируйте бинарный файл `/bin/terraform` на свою локальную машину, используя dive и docker save. Предоставьте скриншоты действий .

Детальный разбор задачи 6:

1. Скачиваю образ `hashicorp/terraform:latest` локально

```
docker pull hashicorp/terraform:latest
```

2. Скачиваю и устанавливаю dive

```
wget https://github.com/wagoodman/dive/releases/download/v0.12.0/dive_0.12.0_linux_amd64.deb
sudo apt install ./dive_0.12.0_linux_amd64.deb
```

3. Анализ образа hashicorp/terraform:latest через dive:

```
dive hashicorp/terraform:latest
```

```
│ Layers │
Cmp   Size  Command
      8.4 MB  FROM blobs
       21 MB  RUN |4 PRODUCT_VERSION=1.14.9 PRODUCT_REVISION=c3c23dd8f1201506dbd21788f482420ee3c1b
      4.9 kB  COPY LICENSE /usr/share/doc/terraform/LICENSE.txt # buildkit
      116 MB  COPY dist/linux/amd64/terraform /bin/terraform # buildkit

│ Layer Details │

Tags:   (unavailable)
Id:     blobs
Digest: sha256:29df493baa13de438d6d2ece3a8333032e0b7b9b9d8cce4ee82194da255f61e1
Command:
ADD alpine-minirootfs-3.23.4-x86_64.tar.gz / # buildkit

│ Image Details │

Image name: hashicorp/terraform:latest
Total Image size: 145 MB
Potential wasted space: 508 kB
Image efficiency score: 99 %

Count   Total Space  Path
    2       436 kB  /etc/ssl/certs/ca-certificates.crt
    2        67 kB  /lib/apk/db/installed
    2       4.7 kB  /lib/apk/db/scripts.tar.gz
    2        326 B  /lib/apk/db/triggers
    2        176 B  /etc/apk/world
```

```
│ ● Current Layer Contents │
Permission     UID:GID       Size  Filetree
drwxr-xr-x         0:0     805 kB  ├── bin
-rwxrwxrwx         0:0        0 B  │   ├── arch → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── ash → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── base64 → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── bbconfig → /bin/busybox
-rwxr-xr-x         0:0     805 kB  │   ├── busybox
-rwxrwxrwx         0:0        0 B  │   ├── cat → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── chattr → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── chgrp → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── chmod → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── chown → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── cp → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── date → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── dd → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── df → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── dmesg → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── dnsdomainname → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── dumpkmap → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── echo → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── egrep → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── false → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── fatattr → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── fdflush → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── fgrep → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── fsync → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── getopt → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── grep → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── gunzip → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── gzip → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── hostname → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── ionice → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── iostat → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── ipcalc → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── kbd_mode → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── kill → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── link → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── linux32 → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── linux64 → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── ln → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── login → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── ls → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── lsattr → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── lzop → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── makemime → /bin/busybox
-rwxrwxrwx         0:0        0 B  │   ├── mkdir → /bin/busybox
```

```
^C Quit | Tab Switch view | ^F Filter | Space Collapse dir | ^Space Collapse all dir | ^O Toggle sort order | ^A Added | ^R Removed | ^M Modified | ^U Unmodified | ^B Attributes | ^P Wrap
```

4. Поиском находим нужный файл.

5. Вышел из dive и сохранил образ в архив:

```
docker save hashicorp/terraform:latest -o terraform_image.tar
mkdir terraform_layers && tar -xf terraform_image.tar -C terraform_layers/   # создал папку и  распаковал в нее
ls terraform_layers/
# содержимое
```

```
● vboxuser@KVA:~/shvirtd-example-python$ ls -lh terraform_layers/blobs/sha256/
  total 48M
  -r--r--r-- 1 vboxuser vboxuser  964 Jan  1  1970 1d10ec4073f4ddbdf34a28540a3b9250852ab500cb1c53f68c8bd17d82f474d8
  -r--r--r-- 1 vboxuser vboxuser  918 Jan  1  1970 593ec1064a1ecec0290938e1231d4c1e0950536a75bd1d07159e60aff9945ea5
  -r--r--r-- 1 vboxuser vboxuser 9.5M Jan  1  1970 6428aaed2d226653f7b022f5f889f26d1d1e12cce64f472ab91ec88d4c3d55c9
  -r--r--r-- 1 vboxuser vboxuser 3.8M Jan  1  1970 76cbedd71a05bf6e8513645fbf8bf436d911348bb3f027280fb28a7476ed12d0
  -r--r--r-- 1 vboxuser vboxuser 2.1K Jan  1  1970 7bbc6f499d554c09a64488ba8b9e1000938f68a873ff4d13cadafbb74fd356c4
  -r--r--r-- 1 vboxuser vboxuser  35M Jan  1  1970 dc7f2a66887b7eb99d6f4650163203c43b1badc1e48354de5df13bd64d44ed98
  -r--r--r-- 1 vboxuser vboxuser 4.2K Jan  1  1970 e54ea6b1f53390b341fc4b885cc208ad179cf2028bd5e483e2c7cf06d20eccc6
```

6. Распаковываем из самого большого файла 35 мб (размер архива, разархивированный +- 116 мб как показал dive) наш файл в созданную для этого директорию `terraform_extract`:

```
mkdir terraform_extract
tar -xzf terraform_layers/blobs/sha256/dc7f2a66887b7eb99d6f4650163203c43b1badc1e48354de5df13bd64d44ed98 \
  -C terraform_extract/ \
  bin/terraform
```

```
● vboxuser@KVA:~/shvirtd-example-python$ mkdir terraform_extract
● vboxuser@KVA:~/shvirtd-example-python$ tar -xzf terraform_layers/blobs/sha256/dc7f2a66887b7eb99d6f4650163203c43b1badc1e48354de5df13bd64d44ed98 \
  -C terraform_extract/ \
  bin/terraform
● vboxuser@KVA:~/shvirtd-example-python$ ls -lh terraform_extract/bin/terraform
  -rwxr-xr-x 1 vboxuser vboxuser 111M Apr 20 17:54 terraform_extract/bin/terraform
```

Задача 6.1

Добейтесь аналогичного результата, используя docker cp.
Предоставьте скриншоты действий .

Детальный разбор задачи 6.1:

Для этого нужно создать но не запускать контейнер:

```
docker create --name terraform_temp hashicorp/terraform:latest   # создал временный контейнер из образа без запуска
docker cp terraform_temp:/bin/terraform ./terraform_cp           # скопировал файл из контейнера на хост
ls -lh terraform_cp                                              # проверил искомый файл с именем terraform_cp
docker rm terraform_temp                                         # удалил временный контейнер
```

```
vboxuser@KVA:~/shvirtd-example-python$ docker create --name terraform_temp hashicorp/terraform:latest
043d3a4215c708063fd6fc0a3cc0952bd17a8d40f2fa3b7509865618dcd7c815
vboxuser@KVA:~/shvirtd-example-python$ docker cp terraform_temp:/bin/terraform ./terraform_cp
Successfully copied 116MB (transferred 116MB) to /home/vboxuser/shvirtd-example-python/terraform_cp
vboxuser@KVA:~/shvirtd-example-python$ ls -lh terraform_cp
-rwxr-xr-x 1 vboxuser vboxuser 111M Apr 20 17:54 terraform_cp
vboxuser@KVA:~/shvirtd-example-python$ docker rm terraform_temp
terraform_temp
vboxuser@KVA:~/shvirtd-example-python$
```

Задача 6.2

Предложите способ извлечь файл из контейнера, используя только команду docker build и любой Dockerfile. Предоставьте скриншоты действий.

Детальный разбор задачи 6.1:

`docker build` с флагом `--output` умеет **экспортировать файлы из образа на хост** без запуска контейнера. Подготовлю `Dockerfile` который берёт файл из terraform-образа и выкладывает его наружу.

```
mkdir terraform_build && cd terraform_build
nano Dockerfile
```

Содержимое `Dockerfile`

```
FROM hashicorp/terraform:latest AS source    # Возьмем terraform из официального образа
FROM scratch                                 # пустой образ
COPY --from=source /bin/terraform /terraform # скопируем только нужный файл в чистый образ
```

Собираем:

```
docker build --output=. .    # с флагом output docker не создает новый образ, а выкладывает содержимое финального слоя в текущую папку
ls -lh                       # проверяем
```

```
vboxuser@KVA:~/shvirtd-example-python$ mkdir terraform_build && cd terraform_build
vboxuser@KVA:~/shvirtd-example-python/terraform_build$ nano Dockerfile
vboxuser@KVA:~/shvirtd-example-python/terraform_build$ docker build --output=. .
[+] Building 2.1s (6/6) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 174B
 => [internal] load metadata for docker.io/hashicorp/terraform:latest
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [source 1/1] FROM docker.io/hashicorp/terraform:latest@sha256:1d10ec4073f4ddbdf34a28540a3b9250852ab500cb1c53f68c8bd17d82f474d8
 => => resolve docker.io/hashicorp/terraform:latest@sha256:1d10ec4073f4ddbdf34a28540a3b9250852ab500cb1c53f68c8bd17d82f474d8
 => [stage-1 1/1] COPY --from=source /bin/terraform /terraform
 => exporting to client directory
 => => copying files 115.84MB
vboxuser@KVA:~/shvirtd-example-python/terraform_build$ ls -lh
total 111M
-rw-rw-r-- 1 vboxuser vboxuser  135 Apr 26 17:23 Dockerfile
-rwxr-xr-x 1 vboxuser vboxuser 111M Apr 20 17:54 terraform
vboxuser@KVA:~/shvirtd-example-python/terraform_build$
```

Задача 7

Запустите ваше python-приложение с помощью runC, не используя docker или containerd. Предоставьте скриншоты действий.
В созданную папку `/opt/mycontainer` экспортировал rootfs из образа приложения, проверил

```
cd /opt/mycontainer
docker export $(docker create shvirtd-example-python-web) | sudo tar -xC rootfs
ls rootfs/
```

сгенерировал json config `sudo runc spec`
отредактировать `config.json` в соответствии с `json.tool` секции "args",

```
"args": [
    "uvicorn",
    "main:app",
    "--host",
    "0.0.0.0",
    "--port",
    "8000"
],
```

секцию `"cwd":` на `"/app"` , секцию `"env"`:

```
"env": [

"PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bi
```

```
n:/sbin:/bin",
    "LANG=C.UTF-8",
    "PYTHON_VERSION=3.12.13",
    "TERM=xterm"
],
```

секцию `"terminal": false,`

Данные взяты из `docker inspect shvirtd-example-python-web | python3 -m json.tool | grep -A20 '"Env"'`

запуск: `sudo runc run myapp`

```
vboxuser@KVA:/opt/mycontainer$ sudo nano config.json
vboxuser@KVA:/opt/mycontainer$ sudo runc run myapp
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```